

[Algorithm]

셸 정렬 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



인하대학교
INHA UNIVERSITY

셸 정렬

- 버블 정렬이나 삽입 정렬이 수행되는 과정
 - 이웃하는 원소끼리의 자리 이동으로 원소들이 정렬된다.

- 버블 정렬이 오름차순으로 정렬하는 과정
 - 작은 (가벼운) 숫자가 배열의 앞부분으로 매우 느리게 이동한다.

- 삽입 정렬의 경우 만일 배열의 마지막 원소가 입력에서 가장 작은 숫자일 경우
 - 그 숫자가 배열의 맨 앞으로 이동해야 하므로, 모든 다른 숫자들이 1칸 씩 오른쪽으로 이동해야 한다.

셸 정렬

□ 셸 정렬 (Shell sort)

- 이러한 단점을 보완하기 위해서 삽입 정렬을 이용하여
- 배열 뒷부분의 작은 숫자를 앞부분으로 '빠르게' 이동시키고,
- 동시에 앞부분의 큰 숫자는 뒷부분으로 이동시키고,
- 가장 마지막에는 삽입 정렬을 수행하는 알고리즘

셸 정렬 아이디어

30 60 90 10 40 80 40 20 10 60 50 30 40 90 80

- 먼저 간격 (gap)이 5가 되는 숫자끼리 그룹을 만든다.
 - 1 번째 그룹부터 5 번째 그룹까지 차례로 [30, 80, 50], [60, 40, 30], [90, 20, 40], [10, 10, 90], [40, 60, 80]이다.

h=5

A	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	30					80					50				
2		60					40					30			
3			90					20					40		
4				10					10					90	
5					40					60					80

셸 정렬 아이디어

h=5

A	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	30					80					50				
2		60					40					30			
3			90					20					40		
4				10					10					90	
5					40					60					80

□ 각 그룹 별로 삽입 정렬을 수행한 결과를 1줄에 나열해 보면 다음과 같다.

30 30 20 10 40 50 40 40 10 60 80 60 90 90 80

A	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	30					50					80				
2		30					40					60			
3			20					40					90		
4				10					10					90	
5					40					60					80

그룹별 정렬 후

셸 정렬 아이디어

A 그룹	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	2	30					50	40				80	60			
	3		30	20					40					90		
	4			10					10						90	
	5				40					60						80

그룹별 정렬 후

□ 간격이 5인 그룹 별로 정렬한 결과

- 80과 90같은 큰 숫자가 뒷부분으로 이동하였고,
- 20과 30같은 작은 숫자가 앞부분으로 이동하였다.

셸 정렬 아이디어

- 그 다음엔 간격을 5보다 작게 하여, 예를 들어, 3으로 하여, 3개의 그룹으로 나누어 각 그룹별로 삽입 정렬을 수행한다.
 - 이때에는 각 그룹에 5개의 숫자가 있다.
- 마지막에는 반드시 간격을 1로 놓고 수행해야 한다.
 - 왜냐하면 다른 그룹에 속해 서로 비교되지 않은 숫자가 있을 수 있기 때문이다.
 - 즉, 모든 원소를 1개의 그룹으로 여기는 것이고, 이는 삽입 정렬 그 자체이다.

ShellSort 알고리즘

입력: 크기가 n 인 배열 A

출력: 정렬된 배열 A

1. for each gap $h = [h_0 > h_1 > \dots > h_k = 1]$ // 큰 gap부터 차례로
2. for $i = h$ to $n-1$ {
3. CurrentElement = $A[i]$;
4. $j = i$;
5. while $(j \geq h)$ and $(A[j-h] > \text{CurrentElement})$ {
6. $A[j] = A[j-h]$;
7. $j = j - h$;
8. }
9. $A[j] = \text{CurrentElement}$;
10. }
11. return 배열 A

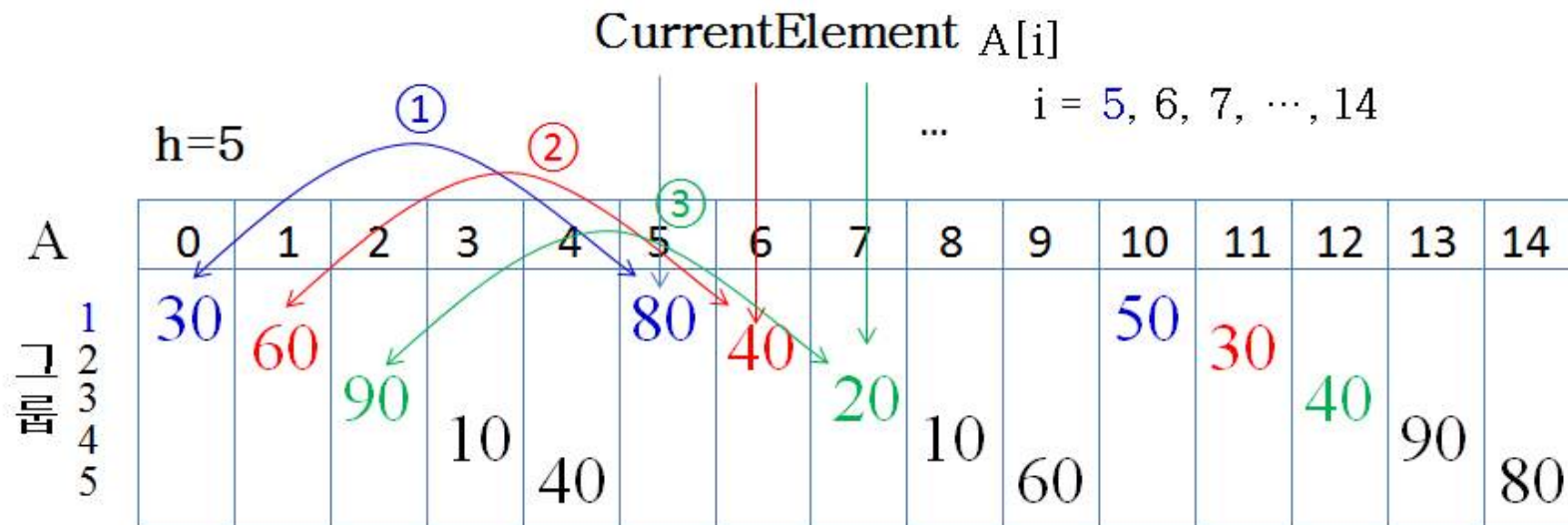
ShellSort 알고리즘

- 셸 정렬은 간격 $[h_0 > h_1 > \dots > h_k=1]$ 이 미리 정해져야 한다.
- 가장 큰 간격 h_0 부터 차례로 간격에 따른 삽입 정렬이 line 2~8에서 수행된다.
- 마지막 간격 h_k 는 반드시 1이어야 한다.
 - 이는 간격에 대해서 그룹별로 삽입 정렬을 수행하였기 때문에, 아직 비교 되지 않은 다른 그룹의 숫자가 있을 수 있기 때문이다.

ShellSort 알고리즘

□ Line 2~8

- for-루프에서는 간격 h 에 대하여 삽입 정렬이 수행되는데, 그룹 별로 삽입 정렬을 수행하지만 자리바꿈을 위한 원소간의 비교는 다음과 같은 순서로 진행된다.
- 즉, line 2의 for-루프가 i 를 h 부터 1씩 증가시켜서 CurrentElement $A[i]$ 를 자신의 그룹에 속한 원소끼리 비교하도록 조절한다.



ShellSort 알고리즘

□ Line 5~7

- while-루프에서 CurrentElement를 정렬이 되도록 앞부분에 삽입한다.
- while-루프의 첫 번째 조건 ($j \geq h$)는 j 가 h 보다 작으면 배열 인덱스 ($j-h$)가 음수가 되어, 배열의 범위를 벗어나는 것을 검사하기 위한 것이다.
- 2번째 조건인 ($A[j-h] > \text{CurrentElement}$)는 ($j-h$)가 음수가 아니면 CurrentElement를 자신의 그룹 원소인 $A[j-h]$ 와 비교하여 크면 line 6에서 $A[j-h]$ 를 h 만큼 뒤로 이동 (즉, $A[j] = A[j-h]$)시킨다.

ShellSort 알고리즘

□ Line 8

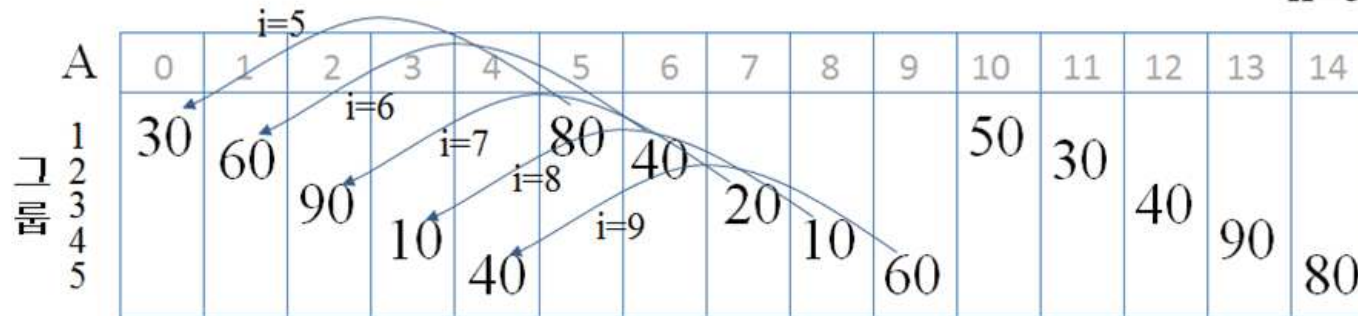
- while-루프의 조건이 '거짓'이 되면 line 8에서 CurrentElement를 $A[j]$ 에 저장한다. 여기서 while-루프의 조건이 '거짓'이 될 때
 - 첫 번째 경우는 $(j-h)$ 가 음수인 경우인데, 이는 $A[j]$ 앞에 같은 그룹의 원소가 없다는 뜻이다.
 - 두 번째 경우는 $A[j-h]$ 가 CurrentElement와 같거나 작은 경우이다.
- 따라서 두 경우 모두 line 8에서 CurrentElement를 $A[j]$ 에 저장하면 알맞게 삽입된다.

ShellSort 수행 과정

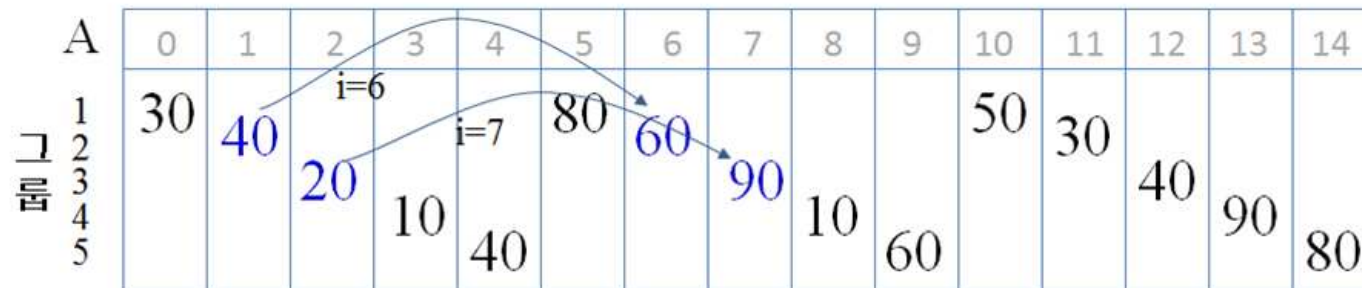
□ 간격이 5일 때, 쉘 정렬의 수행 과정

i = 5, 6, 7, 8, 9일 때

h = 5

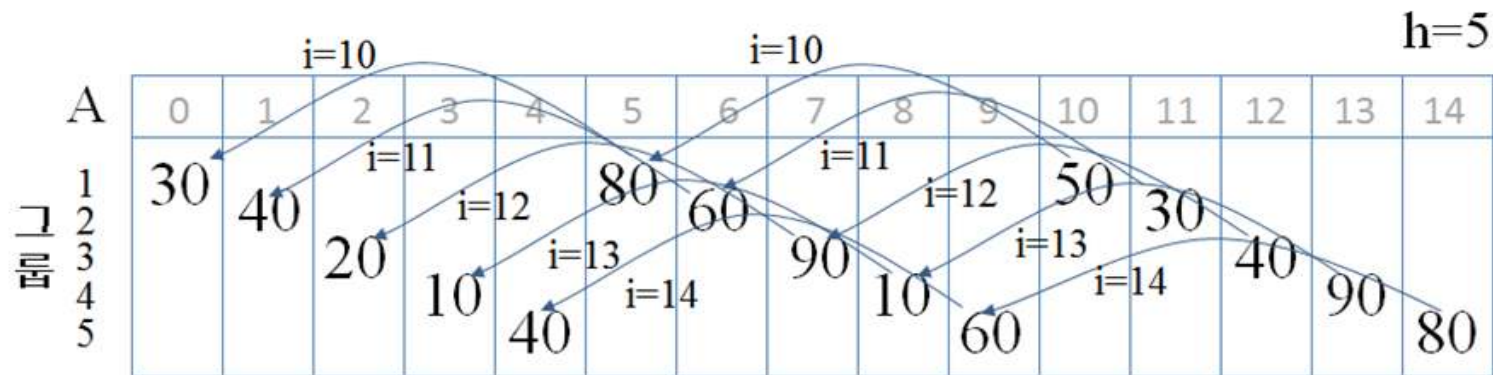


이동 결과

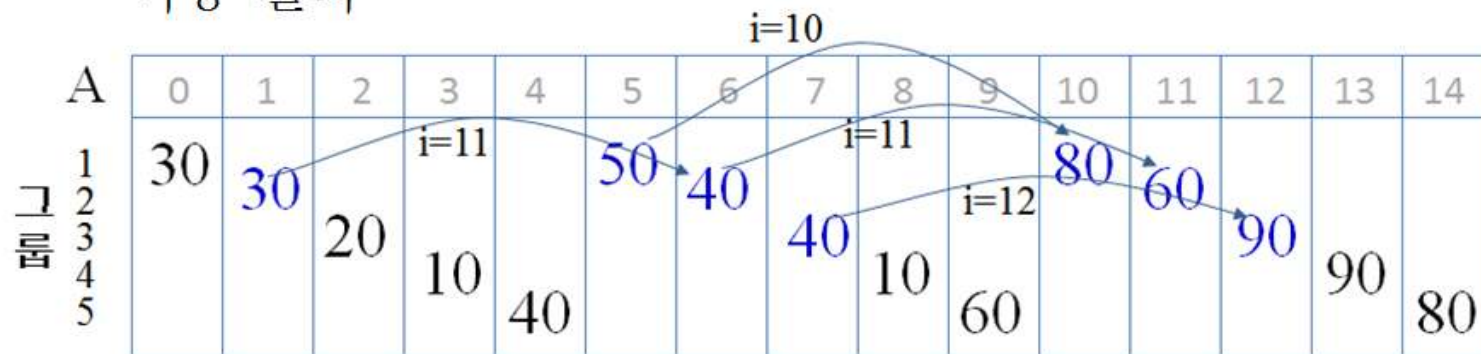


ShellSort 수행 과정

$i=10, 11, 12, 13, 14$



이동 결과



ShellSort 수행 과정

- $h=5$ 일 때의 결과를 한 줄에 나열해보면 다음과 같다.

30 30 20 10 40 50 40 40 10 60 80 60 90 90 80

- 이제 간격을 줄여서 $h=3$ 이 되면, 배열의 원소가 3개의 그룹으로 나누어진다.
 - 그룹1은 0번째, 3번째, 6번째, 9번째, 12번째 숫자이고,
 - 그룹2는 1번째, 4번째, 7번째, 10번째, 13번째 숫자로 구성되고,
 - 마지막으로 그룹3은 2번째, 5번째, 8번째, 11번째, 14번째 숫자이다.

ShellSort 수행 과정

□ 각 그룹 별로 삽입 정렬하면,

그룹1	그룹2	그룹3		그룹1	그룹2	그룹3
30	30	20		10	30	10
10	40	50		30	40	20
40	40	10		40	40	50
60	80	60		60	80	60
90	90	80		90	90	80

□ 각 그룹 별로 정렬한 결과를 한 줄에 나열해보면 다음과 같다. 즉, 이것이 $h=3$ 일 때의 결과이다.

10 30 10 30 40 20 40 40 50 60 80 60 90 90 80

ShellSort 수행 과정

- 마지막으로 위의 배열에 대해 $h=1$ 일 때 알고리즘을 수행하면 아래와 같이 정렬된 결과를 얻는다.
- $h=1$ 일 때는 간격이 1 (즉, 그룹이 1개)이므로, 삽입 정렬과 동일하다.

10 10 20 30 30 40 40 40 50 60 60 80 80 90 90

시간복잡도

- 쉘 정렬의 최악 경우의 시간복잡도는 $O(n^2)$ 이다.
- 쉘 정렬의 수행 속도는 간격 선정에 따라 좌우된다.
- 히바드(Hibbard)의 간격
 - 2^k-1 (즉, $2^k-1, \dots, 15, 7, 5, 3, 1$)을 사용하면, $O(n^{1.5})$ 이다.
- 많은 실험을 통한 쉘 정렬의 시간복잡도는 $O(n^{1.25})$ 으로 알려지고 있다.
- 쉘 정렬의 시간복잡도는 아직 풀리지 않은 문제
 - 이는 가장 좋은 간격을 알아내야 하는 것이 선행되어야 하기 때문이다.
- 지금까지 알려진 가장 좋은 성능을 보인 간격
 - 1, 4, 10, 23, 57, 132, 301, 701, 1750 (Marcin Ciura)

응용

- 쉘 정렬은 입력 크기가 매우 크지 않은 경우에 매우 좋은 성능을 보인다.
- 쉘 정렬은 임베디드(Embedded) 시스템에서 주로 사용되는데, 쉘 정렬의 특징인 **간격에 따른 그룹 별 정렬 방식**이 H/W로 정렬 알고리즘을 구현하는데 매우 적합하기 때문이다.

셸 정렬 알고리즘 구현

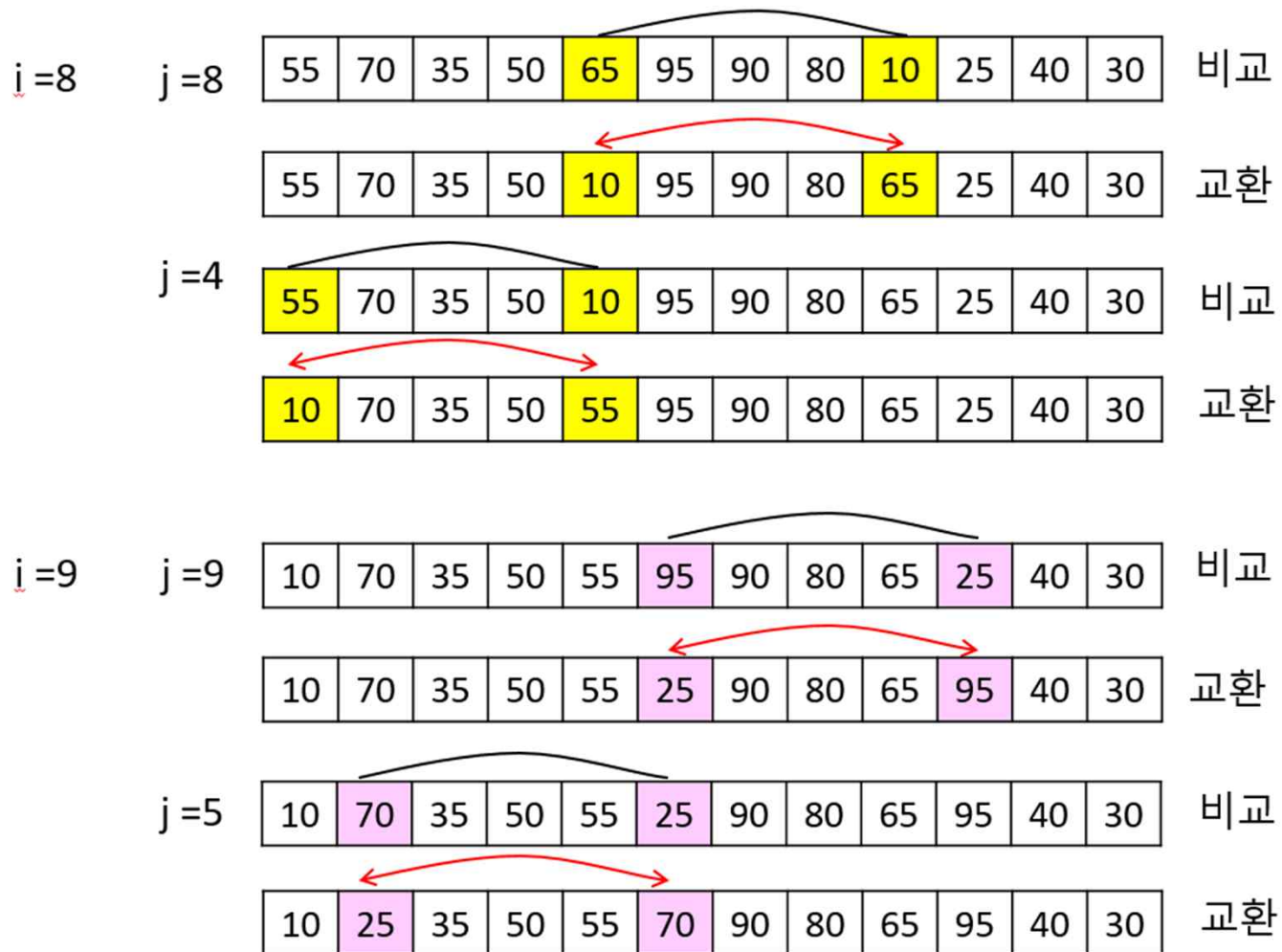
```
def shell_sort(a):
    h = 4      # 3x+1 간격: 1, 4, 13, 40, 121,... 중에서 4 와 1만 사용
    while h >= 1:
        for i in range(h, len(a)): # h-정렬 수행
            j = i
            while j >= h and a[j] < a[j-h]:
                a[j], a[j-h] = a[j-h], a[j]
                j -= h
            h //= 3

a = [54,88,77,26,93,17,49,10,17,77,11,31,22,44,17,20]
print('정렬 전:\t', end='')
print(a)
shell_sort(a)
print('정렬 후:\t', end='')
print(a)
```

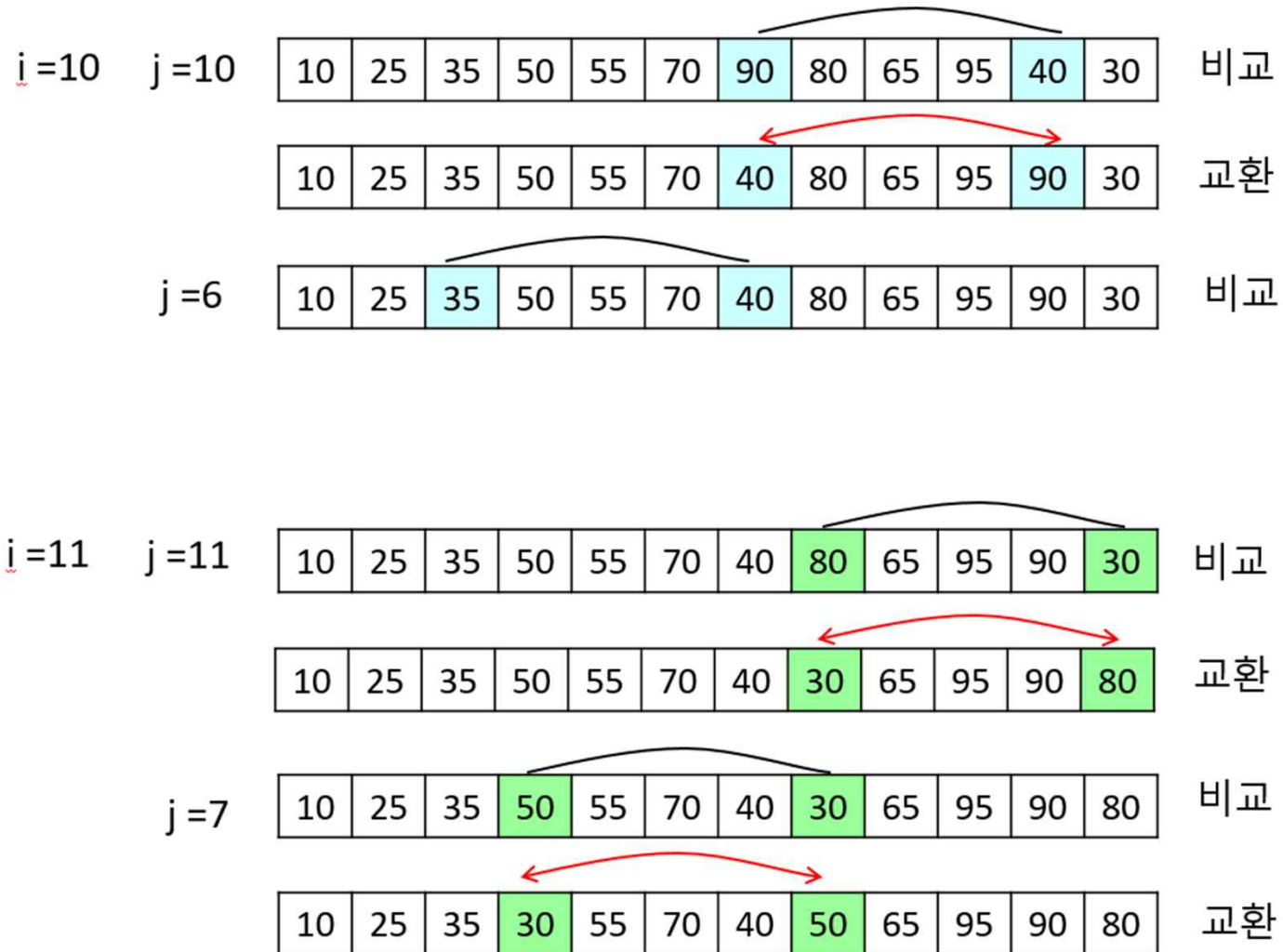
셸 정렬 알고리즘 구현



셸 정렬 알고리즘 구현



셸 정렬 알고리즘 구현





Thank You